Note that you can refine your testing plan as the project development goes on. Keep the change log as follow:

Change Log

Version	Change Date	By	Description
version number	Date of Change	Name of person who made changes	Description of the changes made
1	Feb 27, 2024	Team8	Create testing plan for sprint 1
2	Apr 9, 2024	Ngoc Kien Mai	Release project

Introduction

Scope

Core features:

Account Management:

1. Sign up
2. Log in
3. View profile
4. Modify profile

Course Management:

1. View Course look up page
2. Filter/Search courses based on selected Degree and Term
3. View Course detail
4. Add/Drop Course

Calendar:

1. View Calendar page

Roadmap and Personal Roadmap:

1. View Roadmap page
2. View specific degree Roadmap
3. View Personal Roadmap page
4. Add/Drop courses to Personal Roadmap

Non-functional requirements:

- Average response time less than 300ms

Roles and Responsibilities

Name	Net ID	GitHub username	Role
Quang Anh Le	7900715	AnhLe-Axel	Developer
Jack Nguyen	7920902	ntt2402vn	Developer
Arshdeep	7917921	arsh331	Developer
Ngoc Kien Mai	7876083	maingockien01	Developer

Test Methodology

Test Completeness

Here you define the criteria that will deem your testing complete. For instance, a few criteria to check Test Completeness would be

- 100% back-end code coverage (mandatory for this project), all the back-end source code should be covered by test cases.

Unit tests

- Look up Courses Test:
  - Test Degree.Controller: Function findAll() should return a list of Degree that are available.
  - Test Term.Controller : Function find(tid, department) should return a list of Courses with matched Deperment in that specific Term. Function findAll() should return a list of all Terms.
  -> Acceptance criteria: Result returned should match with provided array JSON . (based on Database implementation)
- Get Courses registered of a User Test:
  - Test User.Course.Controller: function findActive(uid,tid) should return a JSON array of Courses registered with mached User ID and Term ID.
  -> Acceptance criteria: Result returned should match with provided array JSON of Courses. (based on Database implementation)
- Roadmap for degree
  - DegreeController: Get degree by ID with options to include recommended courses

- Degree Service: Get degrees with criteria and options for including recommended courses relations

- Authentication Module:
    - Test auth.controller: Function signup() should create a new user.
- Calendar for the user:
    - Test user.course.controller: Function should return all the courses for a user in a term.
        - Acceptance test: Result should match the test courses.

Integration tests

Total number of test cases: 11

1. term.controller.e2e.ts:
    - Test suite: 'UserCourseController'
        - Test case: 'findAll()': This test case is likely testing the functionality of the 'findAll' method in the 'UserCourseController'. The specifics of the test are not provided in the code excerpt.
        - Test case: 'findCurrent()': This test case is likely testing the functionality of the 'findCurrent' method in the 'UserCourseController'. The specifics of the test are not provided in the code excerpt.
2. term.service.e2e.ts:
    - Test suite: 'TermService'
        - Test case: 'findAll()': This test case is testing the 'findAll' method in the 'TermService'. It expects the method to return an array.
3. userCourse.controller.e2e.ts:
    - Test suite: 'UserCourseController'
        - Test case: 'searchSection': This test case is testing the 'searchSection' method in the 'UserCourseController'. It checks for unauthorized access, allows registered users to access the function, and expects the function to return an array for registered users.
        - Test case: 'searchActive': This test case is testing the 'searchActive' method in the 'UserCourseController'. It checks for unauthorized access, allows registered users to access the function, and expects the function to return an array for registered users.
        - Test case: 'add': This test case is testing the 'add' method in the 'UserCourseController'. It checks for unauthorized access and expects an exception for invalid section id.
        - Test case: 'remove': This test case is testing the 'remove' method in the 'UserCourseController'. It checks for unauthorized access.
4. userCourse.service.e2e.ts:
    - Test suite: 'UserCourseService'
        - Test case: 'findAll()': This test case is testing the 'findAll' method in the 'UserCourseService'. It expects the method to return an array and contain a specific user.
        - Test case: 'find()': This test case is testing the 'find' method in the 'UserCourseService'. It expects the method to find and return a specific user or return null.

- Test case: 'findSection()': This test case is testing the 'findSection' method in the 'UserCourseService'. It expects the method to return an empty array or an array type.
- Test case: 'findActive()': This test case is testing the 'findActive' method in the 'UserCourseService'. It expects the method to return an empty array or an array type.

Load testing:

The Load Testing was tested with 120 users concurrently using the system.

Acceptance criteria: Average response is less than 300ms for an average of 100 users concurrently.

Load Testing includes:

Core features:

Account Management:

5. Sign up request
6. Log in request
7. View profile request
8. Modify profile request

Course Management:

5. View Course look up page request
6. Filter/Search courses based on selected Degree and Term request
7. View Course detail request
8. Add Course request
9. Fetch active registration request (used for later usage of dropping course)
10. Remove/Drop Course request

Calendar:

2. View Calendar page request
3. Fetch active registration request (used for displaying registered courses)

Roadmap and Personal Roadmap:

5. View Roadmap page request
6. View specific degree Roadmap request
7. View Personal Roadmap page request
8. Fetch all courses to display in personal roadmap request
9. Add/Drop courses to Personal Roadmap request

Resource & Environment Needs

Testing Tools

- **Jest** for unit tests in backend & frontend
- **Supertest** for integration tests in backend
- **GitHub Actions** for running tests in CI
- **GitHub Issues** to keep track of bugs, tasks, ...
- **Postman** for testing backend API responses
- **TablePlus/SQL workbench** for troubleshooting database
- **ES-Lint** for linting the code
- **Locust** for load testing

Test Environment

- Windows 10 and above with installed WSL (Ubuntu)
- Docker (integrated with WSL for Windows)
- Docker image:
    - Node 18
    - Global yarn packages required: typescript, @nestjs/cli, concurrently

Terms/Acronyms

Make a mention of any terms or acronyms used in the project

TERM/ACRONYM	DEFINITION
API	Application Program Interface
AUT	Application Under Test